

LAB: Debugging and Reverse Engineering

4/18/26 4:49 PM

Debugging and Reverse Engineering

1. Immunity Debugger – What It Is & Why Use It

What it is: A free Windows debugger designed for malware analysis. It lets you run a program step-by-step under controlled conditions while watching memory, registers, and instructions in real time.

Why use it:

- Shows exactly what the malware does as it executes (dynamic analysis).
- Reveals hidden behavior that static tools (strings, hashes) miss.
- Great for “living off the land” malware that re-uses Windows APIs.

Difference from IDA Free:

- IDA Free** = static disassembly (shows assembly code without running the file).
- Immunity** = dynamic debugging (runs the file and lets you step through assembly line-by-line while it executes).

Best Practice: Use Immunity after static analysis (strings + hash) and before full dynamic tools (Regshot). Always run in a VM with snapshots.

2. The Immunity Debugger Dashboard (Four Main Windows)

When you load any executable (legit or malicious), you see four key sections:

- Disassembly Window (upper-left)**
 - Shows the actual assembly instructions the program will execute.
 - Columns:
 - Address (memory location)
 - Opcode (machine code bytes)
 - Assembly instruction (e.g., MOV EAX, 1)
 - Comment (plain-English explanation when available)
 - What to look for in malware:** API calls (e.g., InternetConnect, RegSetValue, CreateRemoteThread, keybd_event).
- Registers Window**
 - Shows current values in CPU registers (EAX, EBX, ECX, EDX, ESP, EBP, etc.).
 - Why it matters:** Malware often uses registers to store data, pointers, or return values from API calls.
- Hex Dump Window**
 - Shows raw memory in hex + ASCII.
 - Why useful:** Quickly spot strings, URLs, or data the malware is working with.
- Stack Window (lower-right)**
 - Shows the program’s call stack (function calls and local variables).
 - Why useful:** Reveals how functions are called and what data is being passed.

Quick Views to Open:

- View → Log:** Shows loaded DLLs and thread info.
- View → Memory:** Shows memory regions and permissions (look for RWX = read-write-execute, common for injected code).
- View → Executable modules:** Lists all DLLs and executables the program loads.

3. Analyzing a Binary File (Example: ieinstal.exe)

Steps (reusable for any file):

- File → Open → select the executable.
- Let it load (you’ll see the four windows).
- View → Log** → see loaded DLLs and threads.
- View → Memory** → check for suspicious memory blocks.
- View → Executable modules** → list of DLLs the program uses.
- Debug → Restart** to run it again cleanly.

What you learn:

- Which Windows APIs/DLLs the program calls (clue to its capabilities).
- Whether it uses “living off the land” (re-using legitimate Windows components).

4. Analyzing a Malware File (Example: test.exe)

Goal: Identify the malware’s main functions by watching it run.

Steps:

- Run Immunity as Administrator.
- File → Open → select test.exe.
- Immediately check:
 - View → Log** → loaded DLLs.
 - View → Memory** → look for blocks owned by suspicious DLLs (e.g., AcGeneral.dll).
 - View → Executable modules** → confirm loaded modules.

Key Finding Example:

- AcGeneral.dll with Read/Write/Execute permissions → this DLL can record keyboard and mouse input (keylogger/spyware capability).

What to look for in malware analysis:

- Suspicious DLLs (especially those related to keyboard, network, registry, or process injection).
- Memory regions with RWX permissions (classic sign of code injection).
- API calls visible in disassembly (network, file creation, registry changes).

5. Assembly Basics for Malware Analysis (Simple Explanation)

You don’t need to read every line — focus on patterns:

- Common instructions to watch:**
 - MOV / PUSH / POP → moving data around.
 - CALL → calling a function (especially Windows APIs).
 - JMP / JNZ → jumps/conditional logic.
- What to look for in the Disassembly window:**
 - Calls to known bad APIs: InternetOpen, RegCreateKey, CreateFile, WriteProcessMemory, keybd_event.
 - Loops or repeated calls → beaconing or keylogging.
 - String references → URLs, file paths, registry keys.

Why assembly matters:

- Malware hides its true purpose in compiled code.
- Assembly shows the exact steps the malware takes (e.g., “open registry key → write value → hook keyboard”).

Practical Tip: Scroll to the top of disassembly, look at the first few CALL instructions, and check the comment column for plain-English hints.

6. Last Challenge – Specific Checks

Check for data exfiltration:

- Look in **Memory** or **Log** for network-related DLLs or calls.
- In disassembly, search for InternetConnect, HttpSendRequest, or similar.

Check for changes to the Windows registry:

- In **Memory** view, look for registry-related DLLs (advapi32.dll).
- In disassembly, search for RegCreateKey, RegSetValue, RegOpenKey.

Check if AcGeneral.dll is monitoring inputs:

- Confirmed if you see it loaded in **Memory** with RWX permissions.
- In disassembly, look for keyboard/mouse APIs (e.g., SetWindowsHookEx, keybd_event).

Research loaded DLLs:

- Google each DLL name + “malware” or check VirusTotal.
- Common suspicious ones: anything related to keyboard hooks, remote access, or process injection.

Quick Workflow for Any Malware:

- Load in Immunity → check Log, Memory, Executable modules.
- Look for suspicious DLLs and RWX memory.
- Scan disassembly for dangerous API calls.
- Step through (F7 = step into, F8 = step over) if needed.